

Forced Spring–Mass–Damper Simulation: A Computational and Energy-Based Analysis

Documentation for `main.spring_mass_damper_analogy.py`

September 7, 2026

Abstract

This article documents a Python simulation of a single-degree-of-freedom spring–mass–damper system subject to a finite-duration force pulse. The program solves the governing ordinary differential equation numerically, reconstructs the force balance, derives instantaneous powers and energy rates, integrates supplied and dissipated energy, verifies physical balance laws, and presents static and animated visualizations. Particular attention is given to the code’s positive-power convention: a positive component power is power delivered *to* the mass. With the supplied default values, the model is lightly underdamped with damping ratio $\zeta \approx 0.0791$ and natural frequency $\omega_n \approx 3.1623$ rad/s. A headless execution of the script confirms force and power balance at approximately machine precision.

1 Purpose and Scope

The source file `main.spring_mass_damper_analogy.py` is an educational simulation of translational mechanical dynamics and mechanical power flow. It represents a mass connected to a linear spring and viscous damper, with an externally prescribed force. The code is not a symbolic derivation; it computes a sampled numerical trajectory and evaluates all quantities from that trajectory.

The script has eleven logical stages:

1. import numerical, plotting, integration, and animation tools;
2. define physical parameters, initial conditions, and time samples;
3. define the applied pulse force;
4. express the second-order dynamics as a first-order state equation;
5. solve that equation numerically;
6. reconstruct forces and check Newton’s second law;
7. calculate signed instantaneous powers;
8. calculate energy-storage rates and their power balance;
9. calculate stored, supplied, and dissipated energies;
10. calculate damping properties and print numerical diagnostics; and
11. generate static figures and an animated mechanical view.

2 Physical Model

2.1 Coordinates and sign convention

The generalized coordinate $x(t)$ is the displacement of the mass in metres, with positive displacement and applied force directed to the right. Its derivatives are velocity $v(t) = \dot{x}(t)$ in m/s and acceleration $a(t) = \ddot{x}(t)$ in m/s².

All component forces in the program are defined as forces *exerted on the mass*. Thus the spring and damper oppose displacement and velocity respectively:

$$F_{\text{spring}} = -kx, \quad (1)$$

$$F_{\text{damper}} = -cv. \quad (2)$$

Here m is mass in kilograms, c is the viscous damping coefficient in N s/m, and k is spring stiffness in N/m.

2.2 Equation of motion

Summing the externally applied, spring, and damping forces gives Newton's second law:

$$m\ddot{x} = F_{\text{in}}(t) - c\dot{x} - kx. \quad (3)$$

Equivalently,

$$m\ddot{x} + c\dot{x} + kx = F_{\text{in}}(t). \quad (4)$$

This is the forced linear spring-mass-damper equation implemented by the program.

3 Step 1: Imports and Default Parameters

3.1 Imported libraries

`numpy` supplies vectorized arrays and numerical functions such as `linspace`, `sqrt`, `where`, and `max`. `matplotlib.pyplot` creates figures, axes, bars, line plots, annotations, and the rectangular mass. `FuncAnimation` updates the animated figure. From `scipy.integrate`, `solve_ivp` integrates the initial-value problem and `cumulative.trapezoid` numerically integrates power histories into energy histories.

3.2 Model and simulation settings

The program chooses simple SI-unit defaults, listed in Table 1. Initial displacement and velocity are zero, so the system starts at its unstrained equilibrium position and at rest.

The array `t_eval = np.linspace(t_start, t_end, n_points)` contains N equally spaced times. It requests output at intervals of approximately 2.0×10^{-4} s; it does not force the internal Runge-Kutta solver to use that as its integration step size.

4 Step 2: Applied-Force Function

The active input is a rectangular pulse. Its amplitude is computed from the target displacement as

$$F_p = kx_{\text{target}} = (10.0)(0.05) = 0.5 \text{ N}. \quad (5)$$

Table 1: Default parameters in the source file.

Quantity	Symbol	Value
Mass	m	1.0 kg
Damping coefficient	c	0.5 N s/m
Spring stiffness	k	10.0 N/m
Initial displacement	x_0	0.0 m
Initial velocity	v_0	0.0 m/s
Start and end time	t_0, t_f	0.0, 20.0 s
Output samples	N	100001
Target displacement	x_{target}	0.05 m
Pulse duration	T_p	1.0 s

The function `applied_force(time)` implements

$$F_{\text{in}}(t) = \begin{cases} F_p, & t \leq T_p, \\ 0, & t > T_p. \end{cases} \quad (6)$$

The use of `np.where` makes this definition work for either a scalar time supplied by the ODE solver or the full time array supplied later for post-processing. The amplitude is the static force that would hold an ideal undamped spring at x_{target} ; because the pulse ends after one second and the system has inertia and damping, it does not prescribe the exact dynamic displacement.

Commented source lines show an alternative sinusoidal forcing idea, but they have no effect at runtime. The pulse in Equation 6 is the actual forcing used by this version.

5 Step 3: First-Order State Equation

Most numerical ODE routines accept first-order systems. The function `spring_mass_damper(t, state)` therefore stores the state as

$$\mathbf{y}(t) = \begin{bmatrix} x(t) \\ v(t) \end{bmatrix}. \quad (7)$$

It unpacks `state` into `x`, `v`, evaluates Equation 3 for acceleration, and returns

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} v \\ (F_{\text{in}}(t) - cv - kx)/m \end{bmatrix}. \quad (8)$$

In code, the returned list is `[v, acceleration]`. The first element says displacement changes at the current velocity; the second says velocity changes at the acceleration determined by net force divided by mass.

6 Step 4: Numerical Simulation

The call to `solve_ivp` integrates the state equation from t_0 to t_f with initial state `[x0, v0]`. It uses the explicit adaptive Runge–Kutta 4(5) method selected by `method="RK45"`. The requested relative and absolute tolerances, `rtol=1e-9` and `atol=1e-11`, make the integration accuracy stringent for this illustrative problem.

The solver result contains the requested sample times in `solution.t` and a two-row state array in `solution.y`. Consequently,

```
t = solution.t
x = solution.y[0]
v = solution.y[1]
```

extracts the complete time, displacement, and velocity histories. The explicit success check raises `RuntimeError(solution.message)` if the solver reports failure; later calculations therefore do not silently process an incomplete solution.

Acceleration is deliberately reconstructed from the equation of motion rather than numerically differentiating velocity:

$$a = \frac{F_{\text{in}} - cv - kx}{m}. \quad (9)$$

This approach is consistent with the model and avoids amplifying numerical noise through finite differencing.

7 Step 5: Force Reconstruction and Newton's-Law Check

After solving, the script evaluates each force at every sample:

$$F_{\text{spring}} = -kx, \quad (10)$$

$$F_{\text{damper}} = -cv, \quad (11)$$

$$F_{\text{net}} = F_{\text{in}} + F_{\text{spring}} + F_{\text{damper}}. \quad (12)$$

It then computes the residual

$$e_F = F_{\text{net}} - ma. \quad (13)$$

In exact arithmetic $e_F = 0$. Reporting `max(abs(force_balance_error))` gives a direct numerical check that all force signs and the independently reconstructed acceleration agree.

8 Step 6: Instantaneous Power

For a translational force, instantaneous mechanical power is $P = Fv$. The script adopts the convention that positive power is delivered to the mass and negative power is removed from it. This yields

$$P_{\text{spring}} = F_{\text{spring}}v = -kxv, \quad (14)$$

$$P_{\text{damper}} = F_{\text{damper}}v = -cv^2 \leq 0, \quad (15)$$

$$P_{\text{net}} = F_{\text{net}}v = mav, \quad (16)$$

$$P_{\text{input}} = F_{\text{in}}v. \quad (17)$$

The damper always has non-positive delivered power because $c \geq 0$ and $v^2 \geq 0$. It removes mechanical energy regardless of the direction of motion. For plotting and energy accumulation, the source also defines the positive heat-generation rate

$$P_{\text{dissipated}} = -P_{\text{damper}} = cv^2 \geq 0. \quad (18)$$

9 Step 7: Energy Rates and Power Balance

The kinetic and spring potential energies are

$$K = \frac{1}{2}mv^2, \quad (19)$$

$$U = \frac{1}{2}kx^2. \quad (20)$$

Their rates are calculated directly from the states:

$$\frac{dK}{dt} = mav, \quad (21)$$

$$\frac{dU}{dt} = kxv. \quad (22)$$

The sign of dU/dt depends on whether the spring is being loaded or unloaded. The code sums these rates to form the rate of total mechanical energy,

$$\frac{dE}{dt} = \frac{dK}{dt} + \frac{dU}{dt}. \quad (23)$$

Substituting Equation 3 shows the forced-response balance:

$$\frac{dE}{dt} = P_{\text{input}} + P_{\text{damper}} = P_{\text{input}} - P_{\text{dissipated}}. \quad (24)$$

The program tests this relation sample by sample with

$$e_P = \frac{dE}{dt} - P_{\text{input}} - P_{\text{damper}}. \quad (25)$$

10 Step 8: Stored, Dissipated, and Supplied Energy

The sampled values `kinetic_energy`, `spring_energy`, and `mechanical_energy` represent K , U , and $E = K + U$, respectively. The program obtains accumulated damper loss and input work by cumulative trapezoidal integration:

$$E_{\text{diss}}(t) = \int_0^t P_{\text{dissipated}}(\tau) d\tau, \quad (26)$$

$$E_{\text{input}}(t) = \int_0^t P_{\text{input}}(\tau) d\tau. \quad (27)$$

The option `initial=0.0` makes each integral array start at zero and have the same length as `t`. Integrating Equation 24 gives the global balance

$$E(t) + E_{\text{diss}}(t) = E(0) + E_{\text{input}}(t). \quad (28)$$

The code's `energy_balance_error` is the left side of Equation 28 minus its right side. Unlike the force and power residuals, this error includes quadrature error from numerical integration of the sampled power histories.

10.1 Mechanical active and reactive power

For the analogy emphasized by the filename, the damper is the resistive element and the spring and mass are ideal storage elements. The code defines

$$P_{\text{active,mech}} = P_{\text{dissipated}} = cv^2, \quad (29)$$

$$P_{\text{reactive,mech}} = \frac{dU}{dt} + \frac{dK}{dt} = \frac{dE}{dt}. \quad (30)$$

The first quantity is irreversible loss, analogous to resistor power. The second is the instantaneous net rate at which mechanical energy changes in storage. It can be positive or negative and should not be confused with AC reactive power measured in var; it is a time-domain mechanical energy-exchange measure.

11 Step 9: Derived Dynamic Properties

The source calculates standard linear-system quantities:

$$\omega_n = \sqrt{\frac{k}{m}}, \quad (31)$$

$$c_{\text{crit}} = 2\sqrt{km}, \quad (32)$$

$$\zeta = \frac{c}{c_{\text{crit}}}. \quad (33)$$

It classifies the response as underdamped for $\zeta < 1$, critically damped when `np.isclose(zeta, 1.0)` is true, and overdamped otherwise. In the underdamped case it additionally calculates

$$\omega_d = \omega_n \sqrt{1 - \zeta^2}. \quad (34)$$

For critical damping it assigns $\omega_d = 0$; for overdamping it assigns `np.nan`, since an oscillatory damped frequency is not applicable.

With the default values, the printed results are $\omega_n = 3.1623$ rad/s, $c_{\text{crit}} = 6.3246$ Ns/m, $\zeta = 0.0791$, and $\omega_d = 3.1524$ rad/s. The light damping explains why energy transfers between mass and spring after the input pulse while the motion decays gradually.

12 Step 10: Console Diagnostics and Reproducibility

The print block reports parameters, pulse settings, derived properties, initial and final mechanical energy, final cumulative damper loss, and maximum absolute residuals. A headless execution using `MPLBACKEND=Agg python3 main.spring_mass_damper_analogy.py` produced the following balance diagnostics with the default configuration:

Dagnostic	Reported value
Initial mechanical energy	0.00000000 J
Final mechanical energy	0.00000306 J
Dissipated energy	0.04448249 J
Maximum force-balance error	5.551×10^{-17} N
Maximum power-balance error	2.255×10^{-17} W
Maximum energy-balance error	6.843×10^{-8} J

These results support the implementation: force and instantaneous-power checks are at floating-point round-off scale, while the larger but still small energy residual is consistent with cumulative trapezoidal quadrature over sampled values.

13 Step 11: Static Plots

The first figure has four vertically stacked axes sharing a time axis. `constrained_layout=True` lets Matplotlib position titles, labels, and legends without manual spacing.

1. The first panel plots x , v , a , and F_{in} . They have different SI units, so the vertical label deliberately says “State value” rather than implying one common unit.
2. The second panel plots U , K , E , E_{diss} , and E_{input} . Equation 28 is visible by comparing mechanical energy plus dissipated energy with supplied energy.
3. The third panel plots dU/dt , dK/dt , dE/dt , $P_{\text{dissipated}}$, and P_{input} . A horizontal zero line distinguishes power into and out of storage.
4. The fourth panel plots the active damper-loss rate and reactive storage-rate measure. This provides the direct mechanical power analogy.

Every axis receives a two-column legend and a faint grid. The figure title identifies the simulation globally.

14 Step 12: Animation

The animation retains all 100001 simulation samples for analysis but renders at most 600 frames. The array

```
frame_indices = np.linspace(0, len(t) - 1, min(600, len(t)), dtype=int)
```

selects approximately evenly spaced sample indices. This prevents unnecessarily slow display without changing the underlying solution.

The animation contains three panels. The first draws a wall, an equilibrium reference, a zig-zag spring, a damper line, an orange rectangular mass, and a green applied-force arrow. At frame j , the mass location is `equilibrium_x + x[index]`; the equilibrium offset is only a drawing coordinate, not an addition to the physical displacement. The spring coordinates are regenerated each frame to span the changing wall-to-mass distance. The input arrow length is proportional to F_{in}/F_p and is protected against division by zero with `max(pulse_force_amplitude, 1e-12)`.

The second panel updates bars for spring-energy rate, kinetic-energy rate, dissipation rate, and input power. The third updates bars for spring potential energy, kinetic energy, cumulative loss, and cumulative input work. Limits are computed from maxima over the full simulation before animation begins, so bars do not rescale from frame to frame. The `animate(frame)` callback changes graphical artists and returns them. `FuncAnimation` calls that callback repeatedly, using an interval based on the time spacing between displayed frames so that animation time approximately follows simulation time. Finally, `plt.show()` opens both the static and animated figures in an interactive Matplotlib backend.

15 Interpretation and Limitations

The model assumes a linear spring, linear viscous damping, constant mass, one-dimensional translation, and an ideal force source. It omits nonlinear stiffness, Coulomb friction, travel limits, gravity, actuator dynamics, sensor noise, and numerical-event handling at the force discontinuity. The pulse boundary is defined inclusively at $t = T_p$; adaptive integration near that discontinuity may merit an explicit segmented integration strategy in higher-accuracy or production studies.

The quantities E_{diss} and E_{input} are numerical integrals based on exported samples rather than solver-internal quadrature. Increasing output density reduces trapezoidal-integration error but increases memory and plotting cost. The selected 100001 samples are appropriate for a clear smooth visualization of this small educational model.

16 Conclusion

The script implements a coherent mechanical-energy workflow: it solves the forced motion, derives forces from the same governing equation, converts force–velocity products into signed power, integrates power into energy, and quantitatively checks the resulting conservation law with damping and input work included. Its numerical diagnostics demonstrate that the stated sign conventions and equations are mutually consistent for the provided pulse-driven, underdamped example.

References

- [1] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [2] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.